INFO7375 BRANDING & AI · ASSIGNMENT 3

Madison Automation Agent Data Pipeline

An n8n workflow that collects the marketing campaign, ad-copy and industry-brief data the Madison Action Layer needs, then cleans and checks it.

285	94.4%	3	100%
clean records	complete records	sources · 6 feeds/datasets	title, date & source present

1 Project & pipeline overview
2 Data Inventory
3 Quality numbers
4 Data quality checks built in
5 Data dictionary
6 Setup Guide
7 Common errors & fixes
8 How this feeds Assignment 4

Author Mayank Bhadrasen · MS Information Systems
Instructor Professor Nina Harris

Workflow Bhadrasen_Mayank_A3_Workflow.json
Data file madison_a3_records.csv
Data collected 2026-10-02 (n8n 2.12.3, self-hosted)
Submitted October 2, 2026

1 Project & pipeline overview

The problem from Assignment 2, and how this data helps solve it.

Problem. Madison's Content Agent generates and quality-scores ad variants, but its output ends in a CSV file and a Discord message. Nothing reaches a CRM or task tool, so people copy approved content into HubSpot and alert their team by hand. Marketers spend about 10 hours a week (26% of their time) on routine manual work like this (DoubleVerify 2025 Global Insights, 1,970 marketers surveyed).

Solution being built. The **Madison Automation Agent** is an Action Layer with three agents: a **Gatekeeper** that validates and dedupes Madison's variants, a **Routing Agent** (Claude) that decides what action to take, and an **Execution Agent** that creates HubSpot tasks and Slack alerts.

What this assignment collects. To test those agents, we need realistic inputs shaped like Madison's output. Each of the three sources supplies one part of that input:

① Ad copy
HuggingFace ad dataset. Gives the *creative*: headline · body · CTA, matching Madison's variant fields.

② Campaign metadata
HuggingFace campaign dataset. Gives the *context*: channel · campaign type · audience · results.

③ Industry briefs
4 marketing RSS feeds. Give *real, dated topics* to use as campaign briefs.

→

n8n pipeline
Map to one 16-column schema → quality checks → CSV + quality report.

→

Assignment 4
Combined into Madison-style payloads to test Gatekeeper → Routing → Execution.

Workflow at a glance (19 nodes in 4 labelled sections)

Section	Nodes	What it does
① **Collect**	Start Collection → 2 HTTP Request (HuggingFace) + 4 RSS Read	Fetches all sources in parallel. Each fetch retries 3× and continues on error.
② **Normalize**	Map Ad Copy · Map Campaigns · Merge RSS Feeds → Map RSS Articles	Maps each source into the shared schema and records a status for each source (available / unavailable).
③ **Quality**	Merge All Sources → Quality Checks & Cleaning	Validates, de-duplicates, standardises dates and flags incomplete fields.
④ **Save**	Records to Rows → Convert to CSV → Save CSV · Build Quality Report → Convert to JSON → Save Report	Writes the clean data file and the quality report.

Collect once, reuse forever. The workflow was run once on 2026-10-02. The saved CSV is the dataset of record for Assignment 4. Re-running is safe and repeatable (fixed dataset offsets, same schema), but RSS feeds will return newer articles.

2 Data Inventory

In the required template format.

Project: Madison Automation Agent: Action Layer for Madison's Content Agent
Problem: AI-generated marketing content never reaches the systems where work happens (CRM, task tools, team chat). This data supplies realistic ad variants, campaign context and briefs to build and test an automated handoff.

Data Collected

Source 1: HuggingFace: `jaykin01/advertisement-copy`

- **Type:** Product ad copy (product, description, full ad text), split by the workflow into headline · body · CTA
- **Amount:** 90 records (100 fetched, 10 duplicates removed)
- **Purpose:** Supplies the *creative* fields Madison produces (`headline` , `body` , `cta`), so Assignment 4 can build realistic variant payloads
- **Collection method:** n8n HTTP Request → HuggingFace datasets-server rows API (no key), rows 0–99
- **Quality:** 74/90 complete records (82.2%). Incomplete records have no recognisable CTA and are flagged `missing_cta`

Source 2: HuggingFace: `federicomoreno/marketing-campaign-performance-200k`

- **Type:** Structured campaign records: company, campaign type, channel, audience, segment, date, conversion rate, ROI, clicks, impressions
- **Amount:** 100 records (2021-01-01 to 2021-04-10)
- **Purpose:** Supplies the *campaign context* the Routing Agent uses to decide actions and owners (channel, type, audience)
- **Collection method:** n8n HTTP Request → HuggingFace datasets-server rows API (no key), rows 0–99 (mirror of a public Kaggle campaign dataset)
- **Quality:** 100/100 complete records (100%)

Source 3: Marketing industry RSS feeds (4 feeds)

- **Type:** Current marketing and martech news articles: title, summary, categories, date, URL
- **Amount:** 95 records (2026-07-22 to 2026-10-02)
  - MarTech (martech.org): **10** articles (2026-09-30 to 2026-10-02)
  - Marketing Dive: **10** articles (2026-09-29 to 2026-10-02)
  - HubSpot Marketing Blog: **50** articles (2026-07-22 to 2026-10-02)
  - Social Media Examiner: **25** articles (2026-08-20 to 2026-10-01)
- **Purpose:** Real, dated marketing topics used as campaign briefs and as context for routing decisions
- **Collection method:** n8n RSS Read node, one per feed, merged, then mapped
- **Quality:** 95/95 complete records (100.0%)

Statistics

- **Total records:** 285 (from 295 collected; 10 removed: 10 duplicates, 0 missing essentials, 0 invalid dates)
- **Clean records:** 269 fully complete (94.4% quality rate)
- **Sources:** 3 (2 HuggingFace datasets + 4 RSS feeds); 6/6 feeds/datasets available on the collection run
- **Ready for:** Assignment 4: combining ad copy + campaign context + briefs into Madison-style payloads to test the Gatekeeper → Routing → Execution agents (see Section 8)

3 Quality numbers

Every number below comes from `madison_a3_quality_report.json`, which the workflow generates on each run.

Source	Status	Received	Kept	Complete	Complete %	Date range
HuggingFace: jaykin01/advertisement-copy	Available	100	90	74	82.2%	2023-09-25 to 2023-09-25
HuggingFace: federicomoreno/marketing-campaign-performance-200k	Available	100	100	100	100%	2021-01-01 to 2021-04-10
RSS: MarTech (martech.org)	Available	10	10	10	100%	2026-09-30 to 2026-10-02
RSS: Marketing Dive	Available	10	10	10	100%	2026-09-29 to 2026-10-02
RSS: HubSpot Marketing Blog	Available	50	50	50	100%	2026-07-22 to 2026-10-02
RSS: Social Media Examiner	Available	25	25	25	100%	2026-08-20 to 2026-10-01
Total		**295**	**285**	**269**	**94.4%**	

All sources were available on this run.

Quality checklist results

Records complete (target ≥ 80%)	**94.4%**
Title present	100%
Date present	100%
Source name + URL present	100%
Dates in YYYY-MM-DD	100%
Duplicates remaining	0

Records removed

Duplicates	10
Missing title / date / source	0
Invalid or implausible date	0
Total removed	**10**

Quality flags on kept records

Flag	Records	Meaning
`date_from_dataset_release`	90	Ad dataset has no per-row date; the HuggingFace dataset's last-updated date (2023-09-25) is used. Informational, does not make a record incomplete.
`missing_cta`	16	Ad has no recognisable closing call to action. Counted as incomplete.
`body_from_product_description`	10	Ad was only headline + CTA; the dataset's product description is used as the body. Informational.

Known limitations (stated openly)

- **Ad dates:** the ad dataset has no per-row date, so all ads use the dataset's release date and are flagged.
- **Campaign data is synthetic** (a public practice dataset), which suits testing routing logic but not drawing market conclusions. Neither HuggingFace dataset states a license; both are used for coursework only.
- **RSS summaries** come from publishers' feed excerpts, and some end in "[…]" where the publisher truncates them.

4 Data quality checks built in

These checks run inside the workflow on every run, not by hand afterwards.

Check	Where	How it works
Source health	Every fetch node	Retries 3× (2 s apart), then **continues on error**. Map nodes turn a failed or empty response into a `Source unavailable: <name>` status, written to the quality report.
Empty-record check	Quality Checks & Cleaning	Drops any record with no title, no date or no source. These three fields are present in 100% of output rows.
Date validation	Quality Checks & Cleaning	Parses every date and rejects anything unparseable, before 2000 or in the future. Output is always `YYYY-MM-DD`.
Duplicate removal	Quality Checks & Cleaning	Articles: same URL. Ads: same product (the dataset repeats ads with shuffled sentences). Campaigns: same campaign ID.
Text cleaning	Map nodes + Quality step	Strips HTML and escaped entities, removes WordPress “The post … appeared first on …” footers, collapses whitespace, trims all fields.
Completeness flags	Quality Checks & Cleaning	Per-source required fields. Missing ones are named in `quality_flags` (e.g. `missing_cta`) and `is_complete` is set to FALSE. Nothing is silently filled in.
Provenance notes	Map Ad Copy	When a value is derived rather than read directly, the record says so: `date_from_dataset_release`, `body_from_product_description`.
Run report	Build Quality Report	Recalculates every number in this document on each run: per-source counts, completeness, removal breakdown, flag counts.

Handling a slow or empty source

Every fetch node is set to **Retry on Fail (3 tries, 2 s wait)**, **On Error → Continue** and **Always Output Data**. If a source still fails or returns nothing, its Map node outputs a status item instead of records, for example:

```
{ "source_name": "RSS: MarTech (martech.org)", "status": "unavailable",
  "message": "Source unavailable: RSS: MarTech (martech.org) — 403 Forbidden", "records_received":
0 }
```

The other sources keep running, and the message appears in `unavailable_sources` in the quality report. This was tested by simulating a timeout (HuggingFace ads), an empty response (campaigns) and a 403 error (Marketing Dive) in the same run. All three were logged and the run still finished with the remaining data. It also happened for real: during a test run on 2026-10-02, MarTech's Cloudflare returned **429 Too Many Requests**. The workflow logged `Source unavailable: RSS: MarTech (martech.org) — Status code 429` and finished with 275 clean records from the other five feeds (Demo walkthrough, screenshot 4).

5 Data dictionary

`madison_a3_records.csv` : 285 rows × 16 columns, UTF-8, one row per record. Rows are ordered by source type, newest first. “Filled” is the share of rows with a value; empty values are expected where a field doesn't apply to that source (e.g. CTA for articles).

Column	Meaning	Example	Filled
`record_id`	Unique ID: prefix shows the source (CMP, AD, MT, MD, HS, SME)	CMP-0001	100.0%
`source_name`	Exact source the record came from	RSS: Marketing Dive	100.0%
`source_type`	campaign_metadata · ad_copy · industry_article	ad_copy	100.0%
`title_or_headline`	Campaign title, ad headline, or article title	Discover Harem Pants!	100.0%
`body_text`	Campaign summary, ad body, or article summary (plain text, ≤ 500 chars)	Unique, stylish bohemian vibes…	100.0%
`cta`	Call to action (ads only)	Limited stock - shop now!	26.0%
`brand_or_product`	Company (campaigns) or product (ads)	Harem pants	66.7%
`channel`	Delivery channel	Google Ads	68.4%
`campaign_type`	Campaign category	Email	100.0%
`audience`	Target audience · customer segment (campaigns)	Men 18-24 · Health & Wellness	35.1%
`tags`	RSS categories or ad hashtags	B2B marketing; Video marketing	33.7%
`published_date`	Date of the record, always YYYY-MM-DD	2026-09-28	100.0%
`source_url`	Link to the article or dataset page	https://martech.org/…	100.0%
`collected_at`	UTC timestamp of the workflow run	2026-09-29T05:03:46Z	100.0%
`is_complete`	TRUE when every required field for that source type is filled	TRUE	100.0%
`quality_flags`	Why a record is incomplete, or notes on how a value was derived	missing_cta	31.6%

Required fields per source type (used for `is_complete`)

`ad_copy`	title_or_headline, body_text, cta, brand_or_product
`campaign_metadata`	title_or_headline, body_text, channel, campaign_type, audience
`industry_article`	title_or_headline, body_text, source_url
All records	source_name, published_date (valid), title_or_headline. Records missing these are removed, not just flagged.

6 Setup Guide

Written so a classmate can reproduce the dataset from scratch, in about 10 minutes.

What you need first

n8n 2.x	Either self-hosted (this project runs on a Hostinger VPS, n8n 2.12.3) or local: `npm install -g n8n` (needs **Node.js 22.16+**)
Credentials / API keys	**None.** All sources are public and need no login: the HuggingFace datasets-server API and RSS feeds. The workflow contains no secrets, tokens or webhook URLs.
Files	`Bhadrasen_Mayank_A3_Workflow.json`
Internet access	To `datasets-server.huggingface.co`, `martech.org`, `marketingdive.com`, `blog.hubspot.com`, `socialmediaexaminer.com`

Run it

1. **Start n8n.** Local: run `n8n start` in a terminal, then open your browser to `http://localhost:5678`. Self-hosted: open your n8n URL. Create the owner account on first launch.
2. **Import the workflow.** *Overview* → *Create workflow*, then the ⋯ menu (top right) → **Import from File…** → choose `Bhadrasen_Mayank_A3_Workflow.json`. You'll see 19 nodes in four coloured sections.
3. **(Optional) enable disk copies.** n8n 2.x only writes files inside `~/.n8n-files`. Create it once: local `mkdir -p ~/.n8n-files`. Docker/Hostinger: `docker exec -it <container> mkdir -p /home/node/.n8n-files`. Then change the path in both **Save … to Disk** nodes to match (local: `/Users/<you>/.n8n-files/…`). You can skip this step; step 5 works without it.
4. **Run.** Click **Execute workflow**. It finishes in about 10–20 seconds and every node turns green. Item counts appear on each connection.
5. **Download the outputs.** Open **Convert to CSV** → output panel → **Download** (`madison_a3_records.csv`). Do the same on **Convert Report to JSON** (`madison_a3_quality_report.json`).
6. **Check the run.** Open the report and confirm `unavailable_sources` is empty. If a feed is listed there, see Section 7 and re-run later. The rest of the data is still valid.

Changing what is collected

Volume: change `length` (max 100) or `offset` in the two HuggingFace HTTP nodes' query parameters. Feeds: replace the URL in an RSS node, but keep the node name, or update the `FEEDS` list at the top of **Map RSS Articles** to match.

7 Common errors & fixes

Problems seen while building and testing this workflow, and how to resolve them.

Symptom	Fix
RSS node returns **403 Forbidden** or **429 Too Many Requests** (most often MarTech, which sits behind Cloudflare)	Nothing breaks: the node continues on error and the report lists `Source unavailable: RSS: MarTech…`. Wait 10–15 minutes and re-run. Avoid running the workflow many times in a row.
HuggingFace HTTP node times out or returns a 5xx error	The node retries 3 times, 2 seconds apart. If it still fails, the source is logged as unavailable. Check https://status.huggingface.co and re-run later.
Save CSV to Disk shows *"The file … is not writable"* or *ENOENT*	n8n 2.x only allows file writes inside `~/.n8n-files`, and the folder must already exist. Create it (Docker: `docker exec -it <n8n-container> mkdir -p /home/node/.n8n-files`) or download the file from the **Convert to CSV** node instead. The run still completes either way.
Import fails, or nodes show *"Install this node to use it"*	The workflow was built for n8n **2.12.3**. Update n8n (`npm install -g n8n@latest`) and import again.
`n8n start` fails with a Node.js version error (local install)	n8n 2.x requires **Node.js 22.16 or newer**. Check with `node -v` and install the current LTS from nodejs.org.
Emoji or "·" look garbled when the CSV is opened in Excel	The file is UTF-8. In Excel use **Data → From Text/CSV → File origin: UTF-8**, or open it in Google Sheets / Numbers.
Map RSS Articles shows *"Referenced node doesn't exist"*	An RSS node was renamed. The four RSS node names must stay exactly `RSS: MarTech`, `RSS: Marketing Dive`, `RSS: HubSpot Marketing Blog`, `RSS: Social Media Examiner`.

8 How this feeds Assignment 4

From collected records to test payloads for the Action Layer.

Assignment 4 combines one record from each source into a Madison-style variant event, the exact input schema defined in the Assignment 2 PRD:

```
{ "event_id": "evt_A4_001",            "source_agent": "madison.content_agent",
  "brief_id": "MT-0003",                 // ③ industry brief (RSS record_id)
  "variant_id": "AD-0013",               // ① ad copy
  "headline": "Upgrade your denim game!", "body": "Introducing Zipper-detail Jeans…",
  "cta": "Get yours today, stand out in the crowd.",
  // ② campaign context
  "channel": "Google Ads", "campaign_type": "Email", "audience": "Men 18-24 · Foodies",
  "popper_pass": true, "created_at": "2026-09-28" }
```

- **Gatekeeper tests:** the 16 ads flagged `missing_cta` are natural failing inputs for the PRD metric "0 quality-gate violations in 10 deliberately failing inputs".
- **Routing tests:** channel, campaign type and audience give the Routing Agent (Claude) varied cases for choosing HubSpot task owners and Slack alerts.
- **Field-mapping tests:** the consistent 16-column schema supports the PRD metric "0 field-mapping errors in 50 test runs".

Sample records

record_id	AD-0001
title_or_headline	Discover Harem Pants!
body_text	Unique, stylish bohemian vibes with a dropped crotch & loose legs. Comfy meets chic - elevate your wardrobe.
cta	Limited stock - shop now!
brand_or_product	Harem pants
channel	—
campaign_type	Product ad
audience	—
published_date	2023-09-25
is_complete	TRUE
quality_flags	date_from_dataset_release

record_id	CMP-0100
title_or_headline	Display campaign by TechCorp on Facebook
body_text	Targets All Ages in Miami (Spanish) for 60 days. Conversion rate 9%, ROI 5.58, acquisition cost $9,022.00, 521 clicks from 2572 impressions, engagement score 4/10.
cta	—
brand_or_product	TechCorp
channel	Facebook
campaign_type	Display
audience	All Ages · Tech Enthusiasts
published_date	2021-04-10
is_complete	TRUE
quality_flags	—